

Assignment 3 (Weeks 5 and 6)

Name-Krish Agrawal

Roll no.-240559

Part I: Concepts & Theory (Short Answer)

Q 1) Ans . (i) **Atomicity** (All or Nothing) - A transaction is completed entirely or not at all. If any step fails, all changes are undone.

Example: During a bank transfer of ₹500, if the money is deducted from Account A but cannot be added to Account B due to an error, the deduction is rolled back.

(ii) **Consistency** - A transaction brings the database from one valid state to another while maintaining all rules and constraints.

Example: After transferring ₹500, the total balance of both accounts remains the same, ensuring data consistency.

(iii) **Isolation** - Multiple transactions execute independently without interfering with each other.

Example: Two users transferring money simultaneously get correct results.

(iv) **Durability** - Once a transaction is committed, its changes are permanently stored, even if the system crashes afterward.

Example: After a successful bank transfer and COMMIT, the updated balances remain saved even if the database server crashes immediately.

Q 2) Ans. If the database crashes after the first UPDATE but before COMMIT, the transaction is not completed. Since it has not been committed, the database performs a ROLLBACK during recovery. As a result the deduction of ₹500 from Account 101 is undone , Account 202 does not receive ₹500 so both accounts retain their original balances.

COMMIT: Saves all changes permanently. Used when the transaction is successful.

ROLLBACK: Undoes all changes. Used when an error occurs or before a transaction is committed.

Q 3) Ans. The concurrency phenomenon is Non-Repeatable Read because User A reads different values for the same data within the same transaction after User B updates and commits.

ACID Property: Isolation

Transaction Concept: Transaction Isolation Levels (e.g., Repeatable Read or Serializable) are used to prevent this problem.

Q 4) Ans. A View is a virtual table created from the result of an SQL query.

Advantages:

- Simplifies complex queries.
- Improves security by showing only selected data.

No a view can not always updated. A view is updatable only if it is based on a single table, does not use GROUP BY, DISTINCT, aggregate functions, or joins, and includes the primary key.

Q 5) Ans. A CTE is a temporary named result set defined using the WITH clause.

Difference from a subquery:

- A CTE is named and can be reused within the same query, making it easier to read.
- A subquery is written inside another query and cannot be reused.

Types of CTE:

(i) Non-recursive CTE: Executes once.

(ii) Recursive CTE: Refers to itself repeatedly until a condition is met.

Use case of Recursive CTE: Retrieving hierarchical data (e.g., employee-manager or folder structures).

Q 6) Ans. A Stored Procedure is a precompiled collection of SQL statements stored in the database.

Advantages:

- Improves performance because it is precompiled.
- Increases security by allowing controlled access to database operations.

Part II: Practical SQL Application

Q 7) Ans. Create a View

```
CREATE VIEW SalesEmployees AS  
  
SELECT EmpID, Name, Salary  
  
FROM Employees  
  
WHERE Department = 'Sales' ;
```

Q 8) Ans. Execute as a Single Transaction

```
BEGIN ;  
  
UPDATE Employees  
  
SET Salary = Salary + 5000  
  
WHERE Name = 'Eve' ;  
  
COMMIT ;
```

Q 9) Ans. CTE Query

```
WITH AvgSalary AS (  
  
    SELECT AVG(Salary) AS AvgSal  
  
    FROM Employees  
  
)  
  
SELECT Name, Salary  
  
FROM Employees, AvgSalary  
  
WHERE Salary > AvgSal;
```

Part III: Long Answer / System Design

Q) 10 Ans.

(A) Ans. These operations must be executed as a single transaction so that they either all succeed or all fail. This prevents partial updates, ensuring that inventory, payment, order details, and customer history remain consistent and accurate.

(B) Ans. (i) **Atomicity**: Ensures all four operations complete together or none are applied.

(ii) **Consistency:** Keeps the database in a valid state (correct inventory, payment, and order records).

(iii) **Isolation:** Prevents multiple users' transactions from interfering with each other.

(iv) **Durability:** Once the transaction is committed, the changes are permanent even if the system crashes.

(C) Ans. Problem if Atomicity is not enforced

Suppose the payment is recorded and inventory is reduced, but the system crashes before creating the order. The customer is charged but no order exists, leading to financial loss, customer complaints, and inconsistent database records.

(D) Ans. Purpose of Isolation: Isolation ensures that multiple transactions execute independently without affecting each other.

Concurrency Issue – Dirty Read: A transaction reads uncommitted inventory data from another transaction. If the first transaction is later rolled back, the second transaction has used incorrect data, which may lead to wrong inventory counts or incorrect payment processing.

Phantom Read – A transaction sees different sets of rows when repeated because another transaction inserts or deletes records during execution.